



**CENTRO UNIVERSITÁRIO FAMETRO – UNIFAMETRO
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS - ADS**

RELATÓRIO TÉCNICO

DISCIPLINA: TÉCNICAS DE IMPLEMENTAÇÃO DE BANCO DE DADOS

Airton F. R. Nobre, Eric K. F. Gomes, Ramires Nóbrega Lino, João V. V. Oliveira, José
R. C. Calandrine Neto, Levy R. Pereira

1 - INTRODUÇÃO

A utilização de sistemas informatizados tornou-se fundamental para a gestão eficiente de estabelecimentos comerciais, especialmente em ambientes que demandam controle constante de informações, como restaurantes. Nesse contexto, os bancos de dados desempenham papel essencial no armazenamento, organização e recuperação de informações, permitindo maior segurança, confiabilidade e agilidade nos processos operacionais.

O presente relatório detalha o desenvolvimento do banco de dados para o sistema Restaurante 5 Amigos. O tema foi escolhido por representar um cenário real que possui múltiplas entidades relacionadas, movimentação transacional constante e a necessidade de integridade entre dados operacionais e financeiros. Além disso, o domínio permite aplicar diversos conceitos da disciplina, como relacionamentos 1:N e N:N, integridade referencial, normalização, consultas SQL complexas e análise de desempenho.

A proposta busca representar de forma estruturada o funcionamento de um restaurante, permitindo o gerenciamento de clientes, funcionários, mesas, reservas, produtos do cardápio, pedidos, itens consumidos e pagamentos. Dessa forma, o banco de dados torna-se uma ferramenta capaz de apoiar a organização das informações e contribuir para uma gestão mais eficiente do estabelecimento.

Diferente de simples planilhas eletrônicas, o Sistema Gerenciador de Banco de Dados (SGBD) permite controle de concorrência, integridade referencial, segurança, persistência e consultas complexas envolvendo múltiplas tabelas relacionadas. No contexto do Restaurante 5 Amigos, o MySQL/MariaDB foi utilizado como SGBD por permitir gerenciamento eficiente de dados relacionais, execução de consultas SQL e aplicação de restrições estruturais. A escolha do MySQL/MariaDB se deu por ser um SGBD relacional amplamente utilizado no mercado, possuir suporte a integridade referencial, índices, constraints, triggers e otimização de consultas, além de apresentar facilidade de utilização em ambientes acadêmicos.

Objetivos

Geral: Desenvolver uma estrutura de dados relacional eficiente para o controle total de um restaurante, garantindo a integridade, segurança e performance dos dados.

Específicos: Aplicar os princípios da normalização de dados (1FN, 2FN, 3FN), implementar mecanismos de integridade referencial, otimizar consultas através de estratégias de indexação e analisar o processamento interno de consultas utilizando a

álgebra relacional. estratégias de indexação e analisar o processamento interno de consultas utilizando a álgebra relacional.

2 - FUNDAMENTAÇÃO TEÓRICA

2.1 O Modelo Relacional e a Álgebra Relacional

O Modelo Relacional, proposto por Edgar F. Codd em 1970, revolucionou a organização de dados ao introduzir uma estrutura lógica baseada em tabelas (relações), onde os dados são representados por tuplas (linhas) e atributos (colunas) [1]. A formalidade matemática subjacente a este modelo é a Álgebra Relacional, um conjunto de operações (seleção, projeção, união, diferença, produto cartesiano, junção, entre outras) que permitem manipular e consultar dados de forma precisa e rigorosa [2]. O SQL (Structured Query Language), embora seja uma linguagem declarativa de alto nível, tem suas raízes e seu poder expressivo diretamente derivados dos princípios da Álgebra Relacional, atuando como uma interface prática para as operações teóricas.

2.2 Transações e Propriedades ACID

Em sistemas de banco de dados, especialmente em contextos transacionais como o de um restaurante, a confiabilidade das operações é primordial. As Propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade) são um conjunto de garantias que os SGBDs relacionais oferecem para assegurar a integridade dos dados durante as transações [3].

- **Atomicidade:** Garante que uma transação seja tratada como uma única unidade indivisível. Ou ela é totalmente concluída (commit) ou totalmente desfeita (rollback), não havendo estados intermediários.

- **Consistência:** Assegura que uma transação leve o banco de dados de um estado válido para outro estado válido, mantendo todas as regras e restrições de integridade (como chaves estrangeiras e CHECK constraints).

- **Isolamento:** Impede que transações concorrentes interfiram umas nas outras. O resultado final da execução de múltiplas transações simultâneas deve ser o mesmo que se elas tivessem sido executadas sequencialmente.

- **Durabilidade:** Garante que, uma vez que uma transação é confirmada, suas alterações são permanentes e sobreviverão a falhas do sistema (como quedas de energia ou travamentos).

No contexto do Restaurante 5 Amigos, as propriedades ACID são cruciais para operações como o registro de um pedido (que envolve a inserção em pedidos e itens_pedido) ou o processamento de um pagamento, garantindo que todas as etapas sejam concluídas com sucesso ou que nenhuma alteração seja aplicada, prevenindo inconsistências financeiras e operacionais.

2.3 Teorema CAP e a Escolha Arquitetural

O Teorema CAP (Consistência, Disponibilidade e Tolerância a Partições) é um princípio fundamental em sistemas distribuídos, afirmando que é impossível para um sistema de armazenamento de dados distribuído garantir simultaneamente os três requisitos: Consistência (todos os nós veem os mesmos dados ao mesmo tempo), Disponibilidade (o sistema está sempre operacional e responde a requisições) e Tolerância a Partições (o sistema continua operando mesmo que haja falhas de comunicação entre os nós) [4]. Um sistema pode garantir no máximo duas dessas três propriedades. Para o Restaurante 5 Amigos, a escolha de um SGBD relacional como o MySQL prioriza a Consistência e a Disponibilidade (em um ambiente não distribuído ou com replicação síncrona), sacrificando a Tolerância a Partições em cenários de rede complexos. Esta escolha é justificada pela natureza transacional e pela necessidade de alta integridade dos dados financeiros e operacionais do restaurante. No entanto, em um cenário de expansão, uma arquitetura híbrida poderia ser considerada, onde dados menos críticos (como logs de acesso, avaliações de clientes ou cache de cardápio) poderiam ser armazenados em soluções NoSQL (que frequentemente priorizam Disponibilidade e Tolerância a Partições), enquanto os dados transacionais essenciais permaneceriam em um SGBD relacional.

2.4 Normalização de Dados e Prevenção de Anomalias

A Normalização é um processo sistemático de decomposição de relações para reduzir a redundância de dados e evitar anomalias de atualização, inserção e exclusão [5]. O objetivo é organizar as tabelas de forma que cada atributo não-chave dependa da chave primária, da chave primária completa e de nada mais que a chave primária.

- **Anomalias de Inserção:** Ocorrem quando não é possível inserir dados sobre uma entidade sem que existam dados sobre outra entidade. Por exemplo, não seria possível cadastrar um novo produto sem associá-lo a uma categoria existente, se a categoria estivesse embutida na tabela de produtos e não fosse uma entidade separada.

- **Anomalias de Atualização:** Surgem quando a atualização de um dado requer a modificação de múltiplas ocorrências do mesmo valor em diferentes locais, aumentando o risco de inconsistência. Se o nome de uma categoria estivesse repetido em cada produto, uma alteração exigiria múltiplas atualizações.

- **Anomalias de Exclusão:** Acontecem quando a exclusão de dados sobre uma entidade resulta na perda inadvertida de dados sobre outra entidade. Se a exclusão do último produto de uma categoria também removesse a categoria, informações importantes seriam perdidas.

O projeto do Restaurante 5 Amigos foi modelado para atender, no mínimo, à Terceira Forma Normal (3FN), garantindo que:

- *Primeira Forma Normal (1FN):* Todos os atributos são atômicos e não multivalorados (e.g., um único telefone por campo, não uma lista).

- *Segunda Forma Normal (2FN):* Todos os atributos não-chave são funcionalmente dependentes da chave primária completa (especialmente relevante em tabelas associativas como ITEM_PEDIDO).

- *Terceira Forma Normal (3FN):* Não existem dependências transitivas, ou seja, atributos não-chave não dependem de outros atributos não-chave.

Esta abordagem minimiza a redundância e protege a integridade dos dados, tornando o sistema mais robusto e menos propenso a erros.

3 - DESCRIÇÃO DO CENÁRIO ESCOLHIDO

O *Restaurante 5 Amigos* é um estabelecimento fictício que simula um ambiente de atendimento presencial, caracterizado por uma dinâmica operacional complexa que exige um sistema de gerenciamento de banco de dados robusto. O cenário contempla a interação entre clientes, funcionários, mesas, cardápio, pedidos, itens consumidos, pagamentos e reservas, refletindo um fluxo transacional contínuo e a necessidade de alta integridade de dados

O restaurante possui mesas destinadas ao consumo no local, funcionários responsáveis pelo atendimento aos clientes e um cardápio composto por diferentes categorias de produtos, como pratos principais, bebidas, sobremesas e acompanhamentos. Além disso, o estabelecimento oferece a possibilidade de realização de reservas antecipadas.

O processo operacional inicia-se quando um cliente ocupa uma mesa disponível ou realiza uma reserva previamente. Em seguida, um funcionário registra o pedido contendo um ou mais itens do cardápio. Cada produto possui informações específicas, como nome, descrição, categoria, preço e disponibilidade.

O fluxo operacional é delineado pelas seguintes etapas:

1. *Interação Cliente-Mesa-Funcionário*: Clientes podem ocupar mesas disponíveis ou realizar reservas prévias. O atendimento é mediado por funcionários, que registram os pedidos.

2. *Gestão de Pedidos e Itens*: Cada pedido é uma agregação de um ou mais itens do cardápio, estabelecendo um relacionamento muitos-para-muitos (N:N) entre pedidos e produtos, resolvido por uma tabela associativa (ITEM_PEDIDO).

3. *Processamento Financeiro*: A quitação dos pedidos é realizada através de pagamentos, que podem ter diferentes formas. A cardinalidade entre pedido e pagamento é 0:1, reconhecendo que pedidos em estados intermediários (aberto, em preparo, cancelado) podem não possuir um registro de pagamento finalizado.

Durante o atendimento, o pedido permanece associado ao cliente, à mesa utilizada e ao funcionário responsável. Após a inclusão dos itens consumidos, o sistema calcula o valor total do pedido com base nas quantidades solicitadas e nos respectivos preços unitários.

Ao final do atendimento, é realizado o pagamento, que pode ocorrer por diferentes formas, como dinheiro, Pix, cartão de crédito ou cartão de débito. O sistema também mantém o controle dos status dos pedidos, pagamentos e reservas, permitindo o acompanhamento completo das operações realizadas.

A escolha desse cenário foi motivada pela sua proximidade com situações reais encontradas no mercado, possibilitando a aplicação de conceitos importantes da modelagem de bancos de dados relacionais em um contexto prático e de fácil compreensão.

3.1 - Regras de Negócio e Restrições de Integridade

As regras de negócio foram formalizadas para refletir o comportamento esperado do sistema e são intrinsecamente ligadas às restrições de integridade do banco de dados:

- **R1**: Relacionamento Cliente-Pedido (1:N): Um cliente pode realizar múltiplos pedidos ao longo do tempo, mas cada pedido é associado a um único cliente. Esta regra é imposta por uma chave estrangeira (id_cliente em pedidos referenciando clientes).

- **R2**: Pedido Mínimo: Todo pedido deve conter pelo menos um item. Esta restrição é garantida pela lógica de aplicação ou por CHECK constraints e pela estrutura da tabela itens_pedido .

- **R3**: Cálculo Automático de Valor Total: O da soma dos valor_total de um pedido é derivado subtotal de seus itens, sendo calculado automaticamente pelo SGBD através de colunas geradas (GENERATED ALWAYS AS) ou triggers, evitando inconsistências de cálculo.

- **R4:** Disponibilidade de Produtos: Produtos marcados como ‘Indisponível’ no cardápio não podem ser adicionados a novos pedidos, mantendo a coerência operacional.

- **R5:** Políticas de Integridade Referencial: A política padrão para chaves estrangeiras é ON DELETE RESTRICT e ON UPDATE CASCADE . O RESTRICT impede a exclusão de registros pai que possuam dependências em tabelas filhas, protegendo o histórico transacional. O CASCADE garante que atualizações em chaves primárias sejam propagadas automaticamente para as chaves estrangeiras correspondentes, mantendo a consistência dos identificadores.

4 - METODOLOGIA DE DESENVOLVIMENTO

O desenvolvimento do sistema de banco de dados do Restaurante 5 Amigos seguiu um processo metodológico estruturado em três fases distintas, visando garantir a transição suave do domínio de negócio para a implementação técnica:

1. *Modelagem Conceitual:* Identificação das entidades e seus relacionamentos através de reuniões de levantamento de requisitos fictícios, resultando em um Diagrama Entidade-Relacionamento (DER) de alto nível.

2. *Modelagem Lógica:* Mapeamento do DER para o modelo relacional, definindo tabelas, chaves primárias (PK), chaves estrangeiras (FK) e tipos de dados. Nesta fase, aplicou-se rigorosamente as regras de normalização (1FN, 2FN e 3FN).

3. *Modelagem Física:* Implementação do esquema no SGBD MySQL, incluindo a criação de scripts DDL, carga de dados (DML), criação de índices, views e triggers.

5 - MODELO ENTIDADE-RELACIONAMENTO E MODELO RELACIONAL

5.1 Dicionário de Dados

Abaixo, apresenta-se o detalhamento das principais tabelas e suas restrições de integridade, servindo como base para a implementação física.

Tabela	Atributo	Tipo	Restrições	Justificativa
clientes	id_cliente	INT	PK , AUTO_INCREMENT	Identificador único e imutável do cliente.
clientes	cpf	VARCHAR(14)	UNIQUE , NOT NULL	Garante a unicidade do registro por pessoa física.
pedidos	status_pedido	ENUM	NOT NULL , DEFAULT 'Aberto'	Restringe os estados possíveis do pedido para integridade de domínio.
itens_pedido	subtotal	DECIMAL	GENERATED ALWAYS	Garante que o subtotal seja sempre o produto da quantidade pelo preço unitário.
pagamentos	id_pedido	INT	FK , UNIQUE	Garante a cardinalidade 1:1 entre pedido e pagamento finalizado.

A estrutura do banco de dados foi desenvolvida seguindo os princípios do modelo relacional. A modelagem foi elaborada com o objetivo de representar os elementos do ambiente do restaurante e as interações existentes entre eles, garantindo organização, consistência e integridade dos dados.

5.2 Entidades e Atributos Principais

As principais entidades identificadas no sistema são:

- Cliente;
- Funcionário;
- Mesa;
- Produto;
- Pedido;

- Item do Pedido;
 - Pagamento.
- *CLIENTE*: Identificação única, CPF, contato e status.
- *FUNCIONARIO*: Registro de colaboradores e cargos (Garçom, Caixa, etc.).
- *MESA*: Controle de capacidade e status (Livre, Ocupada, Reservada).
- *PRODUTO*: Itens do cardápio com preço e disponibilidade.
- *PEDIDO*: Entidade central que conecta cliente, mesa e funcionário.
- *ITEM_PEDIDO*: Tabela associativa que resolve a relação N:N entre Pedido e Produto.
- *PAGAMENTO*: Registro financeiro vinculado ao pedido.

Cada entidade representa um conjunto específico de informações e possui atributos próprios para caracterizar seus registros.

Os relacionamentos definidos entre as entidades permitem representar as regras de negócio do restaurante. Um cliente pode realizar vários pedidos e reservas, enquanto cada pedido ou reserva pertence a apenas um cliente. Da mesma forma, um funcionário pode atender diversos pedidos, porém cada pedido possui apenas um funcionário responsável.

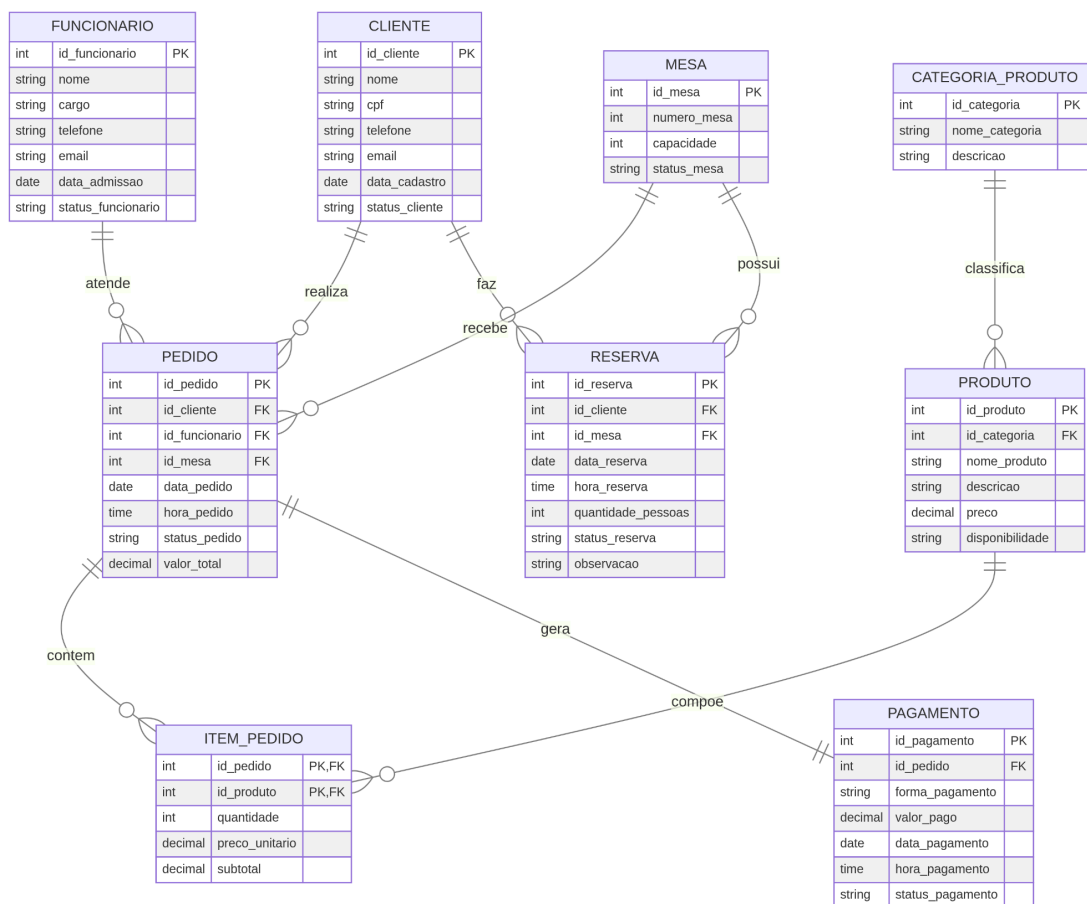
As mesas podem ser utilizadas em diferentes atendimentos ao longo do tempo e também podem estar associadas a diversas reservas em datas distintas. Os produtos do cardápio são organizados em categorias, estabelecendo um relacionamento entre as entidades Categoria de Produto e Produto.

Um dos relacionamentos mais importantes do sistema ocorre entre Pedido e Produto. Como um pedido pode conter vários produtos e um produto pode estar presente em diversos pedidos, foi identificado um relacionamento muitos-para-muitos (N:N). Para resolver essa situação, foi criada a entidade associativa **Item_Pedido**, responsável por armazenar os produtos incluídos em cada pedido, bem como suas quantidades, preços unitários e subtotais.

Por fim, cada pedido possui um pagamento associado, permitindo registrar informações relacionadas à quitação financeira do consumo realizado.

5.3 Diagrama Entidade-Relacionamento (DER)

O diagrama abaixo ilustra a arquitetura do banco de dados, destacando as chaves primárias (PK) e estrangeiras (FK), bem como as cardinalidades.



6 - DESCRIÇÃO DAS TABELAS

6.1 CLIENTE

A tabela CLIENTE é responsável pelo armazenamento dos dados dos consumidores cadastrados no sistema.

Principais campos:

- id_cliente (Chave Primária);
- nome;
- cpf;
- telefone;
- email;
- data_cadastro;

- status_cliente.

Sua principal função é identificar os clientes e permitir a associação de pedidos e reservas a uma pessoa específica.

6.2 FUNCIONÁRIO

A tabela FUNCIONARIO armazena as informações dos colaboradores do restaurante.

Principais campos:

- id_funcionario (Chave Primária);
- nome;
- cargo;
- telefone;
- email;
- data_admissao;
- status_funcionario.

Essa tabela permite identificar qual funcionário realizou determinado atendimento ou operação.

6.3 MESA

A tabela MESA representa as mesas físicas disponíveis no restaurante.

Principais campos:

- id_mesa (Chave Primária);
- numero_mesa;
- capacidade;
- status_mesa.

Sua finalidade é controlar a disponibilidade das mesas, indicando se estão livres, ocupadas, reservadas ou inativas.

6.4 RESERVA

A tabela RESERVA registra os agendamentos realizados pelos clientes.

Principais campos:

- id_reserva (Chave Primária);
- id_cliente (Chave Estrangeira);
- id_mesa (Chave Estrangeira);
- data_reserva;
- hora_reserva;
- quantidade_pessoas;
- status_reserva;
- observacao.

A tabela permite organizar as reservas e evitar conflitos de agendamento.

6.5 CATEGORIA_PRODUTO

A tabela CATEGORIA_PRODUTO é responsável pela classificação dos produtos do cardápio.

Principais campos:

- id_categoria (Chave Primária);
- nome_categoria;
- descricao.

Sua utilização reduz redundâncias e facilita a organização dos produtos.

6.6 PRODUTO

A tabela PRODUTO armazena os itens comercializados pelo restaurante.

Principais campos:

- id_produto (Chave Primária);
- id_categoria (Chave Estrangeira);
- nome_produto;
- descricao;
- preco;
- disponibilidade.

Sua função é manter o cadastro dos produtos disponíveis para venda.

6.7 PEDIDO

A tabela PEDIDO representa os pedidos realizados pelos clientes durante o atendimento.

Principais campos:

- id_pedido (Chave Primária);
- id_cliente (Chave Estrangeira);
- id_funcionario (Chave Estrangeira);
- id_mesa (Chave Estrangeira);
- data_pedido;
- hora_pedido;
- status_pedido;
- valor_total.

Essa é uma das entidades centrais do sistema, pois conecta clientes, funcionários, mesas, itens consumidos e pagamentos.

6.8 ITEM_PEDIDO

A tabela ITEM_PEDIDO resolve o relacionamento muitos-para-muitos entre PEDIDO e PRODUTO.

Principais campos:

- id_pedido (Chave Primária Composta e Chave Estrangeira);
- id_produto (Chave Primária Composta e Chave Estrangeira);
- quantidade;
- preco_unitario;
- subtotal.

Sua função é registrar detalhadamente os produtos incluídos em cada pedido.

6.9 PAGAMENTO

A tabela PAGAMENTO registra as informações financeiras relacionadas aos pedidos.

Principais campos:

- id_pagamento (Chave Primária);
- id_pedido (Chave Estrangeira);
- forma_pagamento;
- valor_pago;
- data_pagamento;
- hora_pagamento;
- status_pagamento.

Essa tabela permite controlar os pagamentos realizados e acompanhar sua situação.

7 - NORMALIZAÇÃO E INTEGRIDADE

O projeto foi estruturado para atender à Terceira Forma Normal (3FN):

1FN: Todos os atributos são atômicos (ex: telefones e e-mails únicos por registro).

2FN: Atributos não-chave dependem da chave primária total (crucial na tabela ITEM_PEDIDO).

3FN: Não existem dependências transitivas entre atributos não-chave.

7.1 Problemas Evitados

Problema	Solução Aplicada
Duplicação de Categorias	Criação da tabela categorias_produto
Pedido sem Cliente Válido	Restrição de Foreign Key (FK)
Inconsistência de Preço	Uso de preco_unitario histórico na venda
Dados Redundantes	Separação de Mesas e Reservas

8 - IMPLEMENTAÇÃO FÍSICA (SQL)

Abaixo, os scripts de criação das tabelas principais, utilizando tipos de dados otimizados e restrições de integridade.

```
-- Criação da Tabela de Produtos com CHECK constraint
CREATE TABLE produtos (
    id_produto INT PRIMARY KEY AUTO_INCREMENT,
    id_categoria INT NOT NULL,
    nome_produto VARCHAR(100) NOT NULL,
    preco DECIMAL(10,2) NOT NULL,
    disponibilidade ENUM('Disponível', 'Indisponível') NOT NULL,
    CONSTRAINT chk_produto_preco CHECK (preco > 0),
    FOREIGN KEY (id_categoria) REFERENCES categorias_produto(id_categoria)
);

-- Tabela Associativa com Coluna Gerada
CREATE TABLE itens_pedido (
    id_pedido INT NOT NULL,
    id_produto INT NOT NULL,
    quantidade INT NOT NULL,
    preco_unitario DECIMAL(10,2) NOT NULL,
    subtotal DECIMAL(10,2) GENERATED ALWAYS AS (quantidade * preco_unitario)
    STORED,
    PRIMARY KEY (id_pedido, id_produto),
    FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido) ON DELETE CASCADE
);
```

8.1 Estratégia de Indexação

Para garantir alta performance, foram criados índices em colunas frequentemente utilizadas em filtros (WHERE) e junções (JOIN):

- *idx_pedido_cliente* : Otimiza a busca de histórico por cliente.
- *idx_reserva_data_hora* : Acelera a verificação de disponibilidade de mesas.

9 - CONSULTAS E ÁLGEBRA RELACIONAL

9.1 Introdução à Álgebra Relacional

A álgebra relacional é uma linguagem formal utilizada para representar operações realizadas em bancos de dados relacionais. Ela serve como base teórica para o funcionamento das consultas SQL e para o processamento interno executado pelos Sistemas Gerenciadores de Banco de Dados (SGBDs).

Enquanto o SQL é uma linguagem declarativa utilizada na prática pelos usuários, a álgebra relacional descreve logicamente as operações sobre conjuntos de dados. Isso permite representar, analisar e otimizar consultas antes de sua execução física no banco.

As principais operações da álgebra relacional utilizadas neste projeto são:

Operação	Símbolo	Função
Seleção	σ	Filtra linhas (tuplas)
Projeção	π	Seleciona colunas (atributos)
Junção	$\bowtie$	Relaciona tabelas
Renomeação	ρ	Renomeia relações ou atributos
Agrupamento	γ	Realiza agregações
Produto cartesiano	$\times$	Combina relações sem condição

9.2 Consulta 1 — Seleção e Projeção

```
```sql
SELECT
 nome_produto,
 preco,
 estoque
FROM produtos
WHERE disponibilidade = 'Disponível'
AND estoque > 0;
```

*Explicação da consulta: Essa consulta retorna os produtos que estão disponíveis para venda e que ainda possuem estoque positivo.*

Ela utiliza:

- Seleção (WHERE) para filtrar registros;
- Projeção (SELECT) para retornar apenas determinadas colunas;
- Condição composta usando operador lógico `AND`.

### - Representação em Álgebra Relacional



```
 π nome_produto, preco, estoque
(
 σ disponibilidade = 'Disponível'
 AND estoque > 0
 (PRODUTOS)
)
```

### Seleção — $\sigma$

$\sigma$  disponibilidade = 'Disponível' AND estoque > 0

### Projeção — $\pi$

$\pi$  nome\_produto, preco, estoque

## 9.3 Consulta 2 — Junção entre tabelas

```
SELECT
 p.id_pedido,
 c.nome,
 p.data_pedido,
 p.valor_total
FROM pedidos p
INNER JOIN clientes c
 ON p.id_cliente = c.id_cliente
WHERE p.valor_total > 100;
```

*Explicação da consulta: Essa consulta retorna os pedidos acima de R\$100,00 juntamente com o nome do cliente responsável.*

Ela utiliza:

- junção entre tabelas;
- seleção;
- projeção;
- condição composta envolvendo relacionamento e filtro numérico.

## - Representação em Álgebra Relacional

```
 π id_pedido, nome, data_pedido, valor_total
(
 (
 σ valor_total > 100
 (
 PEDIDOS $\bowtie$ PEDIDOS.id_cliente = CLIENTES.id_cliente CLIENTES
)
)
)
```

### Seleção — $\sigma$

$\sigma$  valor\_total > 100

### Projeção — $\pi$

$\pi$  id\_pedido, nome, data\_pedido, valor\_total

## 9.4 Consulta 3 — Junção múltipla com condição composta

```
SELECT
 c.nome AS cliente,
 pr.nome_produto,
 ip.quantidade,
 ip.subtotal
FROM clientes c
INNER JOIN pedidos p
 ON c.id_cliente = p.id_cliente
INNER JOIN itens_pedido ip
 ON p.id_pedido = ip.id_pedido
INNER JOIN produtos pr
 ON ip.id_produto = pr.id_produto
WHERE p.status_pedido = 'Pago'
 AND ip.quantidade >= 2;
```

*Explicação da Consulta, ela retorna:*

*\* nome do cliente;*

*\* produto consumido;*

*\* quantidade comprada;*

\* subtotal do item;

considerando apenas pedidos pagos e itens cuja quantidade seja maior ou igual a 2.

Ela demonstra:

- múltiplas junções;
- condição composta;
- relacionamento N:N;
- filtragem lógica;
- projeção.

#### - Representação em Álgebra Relacional

```

π cliente.nome, produtos.nome_produto, itens_pedido.quantidade, itens_pedido.subtotal
(
 σ pedidos.status_pedido = 'Pago'
 AND itens_pedido.quantidade >= 2
 (
 (
 (
 CLIENTES ⋈ CLIENTES.id_cliente = PEDIDOS.id_cliente PEDIDOS
)
 ⋈ PEDIDOS.id_pedido = ITENS_PEDIDO.id_pedido ITENS_PEDIDO
)
 ⋈ ITENS_PEDIDO.id_produto = PRODUTOS.id_produto PRODUTOS
)
)

```

**Junção 1:** CLIENTES ⋈ CLIENTES.id\_cliente = PEDIDOS.id\_cliente PEDIDOS

**Junção 2:** ⋈ PEDIDOS.id\_pedido = ITENS\_PEDIDO.id\_pedido ITENS\_PEDIDO

**Junção 3:** ⋈ ITENS\_PEDIDO.id\_produto = PRODUTOS.id\_produto PRODUTOS

#### Seleção composta

$\sigma$  pedidos.status\_pedido = 'Pago'

AND itens\_pedido.quantidade >= 2

## Projeção

$\pi$  cliente.nome, produtos.nome\_produto, itens\_pedido.quantidade, itens\_pedido.subtotal

## 9.5 Consulta 4 — Agregação e Agrupamento

```
SELECT
 pr.nome_produto,
 SUM(ip.quantidade) AS total_vendido
FROM produtos pr
INNER JOIN itens_pedido ip
 ON pr.id_produto = ip.id_produto
GROUP BY pr.nome_produto;
```

*Explicação da Consulta: Essa consulta calcula a quantidade total vendida de cada produto.*

Ela utiliza:

- junção;
- agrupamento;
- função agregada;
- projeção.

### - Representação em Álgebra Relacional

```
 γ nome_produto, SUM(quantidade) $\rightarrow$ total_vendido
(
 PRODUTOS $\bowtie$ PRODUTOS.id_produto = ITENS_PEDIDO.id_produto ITENS_PEDIDO
)
```

## Junção

Relaciona produtos aos itens vendidos.

### Agrupamento — $\gamma$

$\gamma$  nome\_produto, SUM(quantidade)

Agrupa os registros pelo nome do produto e soma as quantidades vendidas.

A operação de agrupamento pertence à álgebra relacional estendida, utilizada para representar funções agregadas como:

- `SUM`
- `COUNT`
- `AVG`
- `MIN`
- `MAX`

## 9.6 Conclusão

A álgebra relacional permitiu representar formalmente operações realizadas no banco de dados do **Restaurante 5 Amigos**, demonstrando a base teórica por trás das consultas SQL utilizadas no sistema.

As representações desenvolvidas utilizaram seleção, projeção, junção, agrupamento e condições compostas, evidenciando a coerência entre o modelo relacional e as consultas implementadas.

## 10 - PROCESSAMENTO E OTIMIZAÇÃO DE CONSULTAS

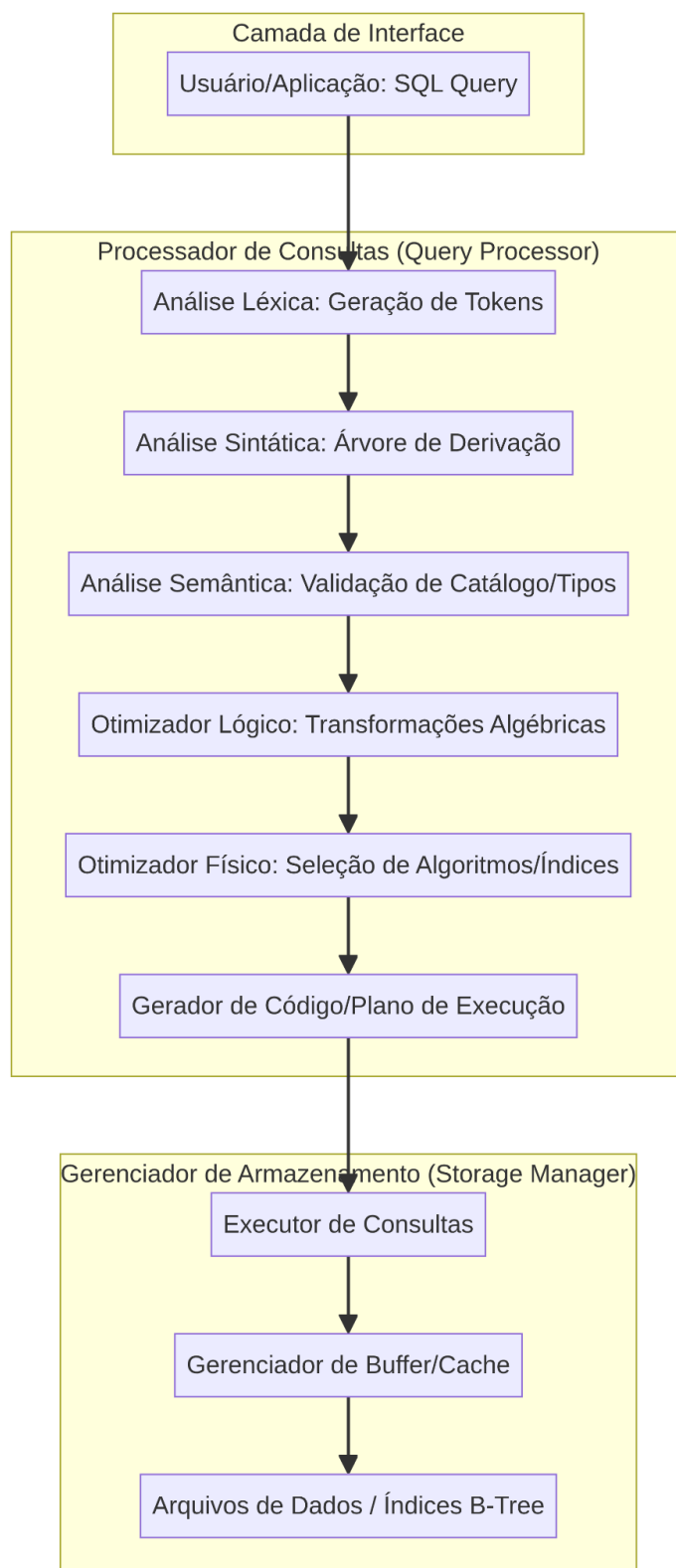
O SGBD segue um fluxo rigoroso e multicamadas para transformar o código SQL declarativo em resultados físicos. Este processo envolve a tradução da consulta em uma representação algébrica e a seleção do plano de execução de menor custo computacional.

### Consulta SQL original escolhida

A consulta escolhida tem como objetivo listar os produtos vendidos em pedidos pagos, exibindo o cliente, o funcionário responsável, o produto, a quantidade, o subtotal e a forma de pagamento.

```
SELECT
 c.nome AS cliente,
 f.nome AS funcionario,
 pr.nome_produto,
 ip.quantidade,
 ip.preco_unitario,
 ip.subtotal,
 pg.forma_pagamento,
 p.data_pedido
FROM pedidos p
INNER JOIN clientes c
 ON p.id_cliente = c.id_cliente
INNER JOIN funcionarios f
 ON p.id_funcionario = f.id_funcionario
INNER JOIN itens_pedido ip
 ON p.id_pedido = ip.id_pedido
INNER JOIN produtos pr
 ON ip.id_produto = pr.id_produto
INNER JOIN pagamentos pg
 ON p.id_pedido = pg.id_pedido
WHERE p.status_pedido = 'Pago'
 AND pg.status_pagamento = 'Aprovado'
 AND p.data_pedido BETWEEN '2025-04-01' AND '2025-04-08'
ORDER BY p.data_pedido, c.nome;
```

Abaixo, uma representação visual clara e objetiva de como funciona o processamento de consultas SQL em um SGBD:



## 10.1 Objetivo da Consulta

Essa consulta retorna os itens vendidos em pedidos pagos e com pagamento aprovado, dentro de um intervalo de datas específico.

Ela envolve:

- junção entre seis tabelas;
- filtros por status do pedido;
- filtros por status do pagamento;
- filtro por intervalo de datas;
- ordenação final;
- seleção apenas das colunas necessárias.

Essa consulta é relevante para análise operacional e financeira, pois permite verificar quais produtos foram vendidos, para quais clientes, por quais funcionários e com qual forma de pagamento.

### - Análise Léxica

Na análise léxica, o SGBD divide a consulta SQL em unidades menores chamadas tokens.

### Exemplos de tokens identificados:

SELECT

c.nome

AS

cliente

FROM

pedidos

p

INNER JOIN

clientes

ON

WHERE

p.status\_pedido

=

'Pago'

AND

BETWEEN

ORDER BY

Nessa etapa, o SGBD reconhece palavras reservadas, identificadores, operadores, literais e símbolos.

Elemento	Tipo léxico
SELECT	Palavra reservada
FROM	Palavra reservada
INNER JOIN	Palavra reservada
c.nome	Identificador de coluna
pedidos	Identificador de tabela
'Pago'	Literal textual
BETWEEN	Operador de intervalo
AND	Operador lógico
=	Operador de comparação

A função dessa fase é verificar se os componentes básicos da linguagem SQL foram reconhecidos corretamente.

#### - Análise Sintática

Na análise sintática, o SGBD verifica se os tokens estão organizados de acordo com a gramática válida do SQL.

A estrutura esperada é:

```
SELECT - colunas
FROM tabela_principal
JOIN tabela_relacionada ON condição
WHERE filtros
ORDER BY colunas;
```



A consulta está sintaticamente correta porque possui:

- cláusula `SELECT`;
- cláusula `FROM`;
- junções com `INNER JOIN`;
- condições de junção com `ON`;
- filtros com `WHERE`;
- ordenação com `ORDER BY`.

Caso houvesse erro, por exemplo:

***SELECT nome FROM WHERE clientes;***

o SGBD rejeitaria a consulta antes da execução, pois a estrutura estaria inválida.

#### - **Análise Semântica**

Na análise semântica, o SGBD verifica se os elementos citados realmente existem e fazem sentido dentro do banco.

Nesta consulta, o SGBD valida:

- se a tabela `pedidos` existe;
- se a coluna `id\_cliente` existe em `pedidos`;
- se a coluna `id\_cliente` existe em `clientes`;
- se `status\_pedido` pertence à tabela `pedidos`;
- se `status\_pagamento` pertence à tabela `pagamentos`;
- se os tipos de dados são compatíveis nas comparações;
- se as chaves estrangeiras permitem as junções propostas.

#### **Exemplo de validação semântica:**

`p.id_cliente = c.id_cliente`

Ambas as colunas são do tipo `INT`, então a comparação é válida.

## 10.2 Representação lógica da consulta

```

π cliente, funcionario, nome_produto, quantidade, preco_unitario, subtotal, forma_pagamento, data_pedido
(
 τ data_pedido ASC, cliente ASC
 (
 σ status_pedido = 'Pago'
 AND status_pagamento = 'Aprovado'
 AND data_pedido >= '2025-04-01'
 AND data_pedido <= '2025-04-08'
 (
 (
 (
 PEDIDOS
 ⋈ PEDIDOS.id_cliente = CLIENTES.id_cliente CLIENTES
)
 ⋈ PEDIDOS.id_funcionario = FUNCIONARIOS.id_funcionario FUNCIONARIOS
)
 ⋈ PEDIDOS.id_pedido = ITENS_PEDIDO.id_pedido ITENS_PEDIDO
)
 ⋈ ITENS_PEDIDO.id_produto = PRODUTOS.id_produto PRODUTOS
)
 ⋈ PEDIDOS.id_pedido = PAGAMENTOS.id_pedido PAGAMENTOS
)
)

```

### - Possível Plano De Execução:

Um possível plano de execução físico do SGBD seria:

1. Acessar a tabela pedidos.
2. Aplicar filtro: *status\_pedido = 'Pago'*  
*data\_pedido BETWEEN '2025-04-01' AND '2025-04-08'*
3. Para cada pedido filtrado, localizar o pagamento correspondente.
4. Aplicar filtro: *status\_pagamento = 'Aprovado'*
5. Juntar pedidos com clientes pelo *id\_cliente*.
6. Juntar pedidos com funcionarios pelo *id\_funcionario*.
7. Juntar pedidos com itens\_pedido pelo *id\_pedido*.
8. Juntar itens\_pedido com produtos pelo *id\_produto*.
9. Projetar apenas as colunas solicitadas.
10. Ordenar o resultado por *data\_pedido* e nome do cliente.
11. Retornar o resultado final.

#### - Criação de índices recomendados:

Para melhorar o desempenho, os seguintes índices são recomendados:

```
CREATE INDEX idx_pedidos_status_data
ON pedidos(status_pedido, data_pedido);

CREATE INDEX idx_pagamentos_status_pedido
ON pagamentos(status_pagamento, id_pedido);

CREATE INDEX idx_itens_pedido_produto
ON itens_pedido(id_produto);

CREATE INDEX idx_clientes_nome
ON clientes(nome);

CREATE INDEX idx_funcionarios_nome
ON funcionarios(nome);
```

Esses índices ajudam em:

- filtros por status;
- filtros por data;
- junções por chave estrangeira;
- ordenações;
- buscas por nome.

### 10.3 Consulta Otimizada Final

Nesta versão, a ordem das junções prioriza primeiro as tabelas diretamente relacionadas ao filtro financeiro e à movimentação:

pedidos → pagamentos → itens\_pedido → produtos → clientes → funcionarios

Isso é adequado porque `pedidos`, `pagamentos` e `itens\_pedido` são as tabelas com maior impacto no volume de dados.

```

SELECT
 c.nome AS cliente,
 f.nome AS funcionario,
 pr.nome_produto,
 ip.quantidade,
 ip.preco_unitario,
 ip.subtotal,
 pg.forma_pagamento,
 p.data_pedido
FROM pedidos p
INNER JOIN pagamentos pg
 ON p.id_pedido = pg.id_pedido
INNER JOIN itens_pedido ip
 ON p.id_pedido = ip.id_pedido
INNER JOIN produtos pr
 ON ip.id_produto = pr.id_produto
INNER JOIN clientes c
 ON p.id_cliente = c.id_cliente
INNER JOIN funcionarios f
 ON p.id_funcionario = f.id_funcionario
WHERE p.status_pedido = 'Pago'
 AND pg.status_pagamento = 'Aprovado'
 AND p.data_pedido BETWEEN '2025-04-01' AND '2025-04-08'
ORDER BY p.data_pedido, c.nome;

```

## 11 - ANÁLISE DE PLANOS DE EXECUÇÃO (EXPLAIN)

Para validar a eficiência, utilizamos o comando EXPLAIN . Ele nos permite identificar se o banco está realizando um Full Table Scan (lento) ou utilizando os índices criados (rápido).

id	select_type	table	type	key	rows	Extra
1	SIMPLE	p	ref	idx_pedido_cliente	12	Using index condition

**Para verificar o plano de execução real no MySQL, pode-se usar:**

```

EXPLAIN
SELECT
 c.nome AS cliente,
 f.nome AS funcionario,
 pr.nome_produto,
 ip.quantidade,
 ip.preco_unitario,
 ip.subtotal,
 pg.forma_pagamento,
 p.data_pedido
FROM pedidos p
INNER JOIN pagamentos pg
 ON p.id_pedido = pg.id_pedido
INNER JOIN itens_pedido ip
 ON p.id_pedido = ip.id_pedido
INNER JOIN produtos pr
 ON ip.id_produto = pr.id_produto
INNER JOIN clientes c
 ON p.id_cliente = c.id_cliente
INNER JOIN funcionarios f
 ON p.id_funcionario = f.id_funcionario
WHERE p.status_pedido = 'Pago'
 AND pg.status_pagamento = 'Aprovado'
 AND p.data_pedido BETWEEN '2025-04-01' AND '2025-04-08'
ORDER BY p.data_pedido, c.nome;

```

O Comando 'EXPLAIN' permite analisar:

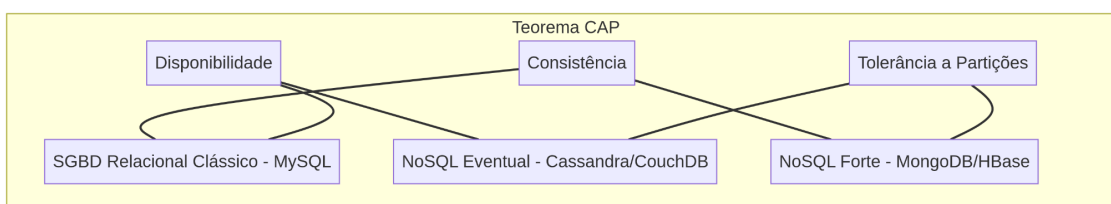
Campo	Significado
<code>table</code>	Tabela acessada
<code>type</code>	Tipo de acesso usado
<code>possible_keys</code>	Índices possíveis
<code>key</code>	Índice escolhido
<code>rows</code>	Estimativa de linhas analisadas
Extra	Operações adicionais, como <code>Using where</code> ou <code>Using temporary</code>

## 12 - COMPARAÇÃO COM NOSQL

Embora o modelo relacional seja ideal para garantir as propriedades ACID em transações financeiras, o Teorema CAP nos mostra que diferentes necessidades exigem diferentes trade-offs arquiteturais.

Em um cenário de escala global para o restaurante, uma arquitetura híbrida seria a recomendação técnica ideal:

- Camada Relacional (MySQL): Responsável pelo núcleo transacional (Pedidos, Pagamentos e Clientes), onde a consistência é inegociável.
- Camada NoSQL (MongoDB/Redis): Utilizada para armazenamento de documentos flexíveis (Logs de acesso, histórico de navegação, avaliações de clientes) e cache de alta performance para o cardápio, priorizando a disponibilidade e escalabilidade.



O banco de dados desenvolvido para o Restaurante 5 Amigos foi modelado com base no modelo relacional, utilizando tabelas, chaves primárias, chaves estrangeiras, restrições de integridade e relacionamentos entre entidades.

Como análise complementar, é importante comparar essa escolha com uma possível solução NoSQL, avaliando se o modelo relacional realmente atende melhor ao cenário do sistema ou se algum módulo poderia se beneficiar de uma abordagem não relacional.

### 12.1 Vantagens e Desvantagens de usar NoSQL nesse contexto

- *VANTAGENS*

- **Flexibilidade de estrutura**

Bancos NoSQL permitem armazenar dados sem um esquema rígido. Isso é útil para informações variáveis, como avaliações, logs, preferências do cliente e conteúdo do cardápio digital.

- **Escalabilidade horizontal**

Soluções NoSQL normalmente são adequadas para crescimento horizontal, distribuindo dados em múltiplos servidores com mais facilidade. Isso pode ser interessante caso o restaurante evolua para uma rede com várias unidades.

- **Boa performance para leituras específicas**

NoSQL pode ser eficiente para consultas simples e rápidas em documentos completos, principalmente quando os dados já estão agrupados no formato em que serão exibidos.

Exemplo: carregar todo o cardápio digital de uma vez.

- **Armazenamento eficiente de logs**

Logs e eventos podem ser gravados em grande volume sem a necessidade de muitas junções. Nesse caso, NoSQL pode ser mais prático e escalável.

- *DESVANTAGENS*

- **Menor controle de integridade relacional**

Como o sistema possui muitos relacionamentos, a ausência de chaves estrangeiras tradicionais poderia gerar inconsistências.

*Exemplo:*

*Pedido referenciando cliente inexistente.*

*Pagamento sem pedido válido.*

*Produto duplicado em diferentes documentos.*

## - Maior redundância de dados

Em NoSQL, é comum duplicar informações para melhorar leitura. Porém, isso pode gerar problemas de atualização.

Exemplo: se o nome de um produto estiver gravado em vários documentos de pedidos, uma alteração exigiria atualização em múltiplos locais.

## - Consultas relacionais mais difíceis

Consultas como esta são naturais em SQL:

```
SELECT
 c.nome,
 p.id_pedido,
 pr.nome_produto,
 ip.quantidade,
 pg.forma_pagamento
FROM pedidos p
JOIN clientes c ON p.id_cliente = c.id_cliente
JOIN itens_pedido ip ON p.id_pedido = ip.id_pedido
JOIN produtos pr ON ip.id_produto = pr.id_produto
JOIN pagamentos pg ON p.id_pedido = pg.id_pedido;
```

*Em NoSQL, essa mesma consulta poderia exigir documentos duplicados, múltiplas buscas ou tratamento adicional na aplicação.*

## - Menor adequação para transações financeiras

O sistema envolve pagamentos, pedidos, valores e fechamento de consumo. Esses processos exigem forte consistência. Bancos relacionais são mais indicados para esse núcleo transacional.

## • Comparação direta entre Relacional e NoSQL

Critério	Banco Relacional	Banco NoSQL
Estrutura dos dados	Rígida e definida por tabelas	Flexível, baseada em documentos, chave-valor, colunas ou grafos
Relacionamentos	Forte suporte com chaves estrangeiras	Relacionamentos geralmente tratados pela aplicação
Consistência	Alta consistência e integridade	Pode priorizar disponibilidade e escalabilidade
Consultas com JOIN	Natural e eficiente com modelagem adequada	Menos indicado para muitos relacionamentos
Transações	Forte suporte	Depende do tipo de banco e implementação
Escalabilidade	Tradicionalmente vertical, embora possa escalar	Forte em escalabilidade horizontal
Melhor uso no projeto	Pedidos, clientes, pagamentos, produtos, reservas	Logs, avaliações, cache, cardápio digital flexível

### 13. Conclusão

O projeto do Restaurante 5 Amigos demonstra uma aplicação sólida dos conceitos de Banco de Dados. A estrutura desenvolvida não apenas armazena informações, mas garante a integridade e a performance necessária para um ambiente real. A integração entre a prática do SQL e a teoria da Álgebra Relacional consolidou o entendimento sobre o processamento interno dos dados.

O banco relacional foi a melhor escolha para o sistema Restaurante 5 Amigos, pois o projeto possui muitos relacionamentos, forte dependência entre entidades e necessidade de integridade dos dados.

As tabelas `clientes`, `pedidos`, `itens\_pedido`, `produtos`, `pagamentos`, `mesas` e `reservas` dependem diretamente umas das outras. Por isso, o uso de chaves primárias, chaves estrangeiras, restrições e normalização torna o modelo mais confiável e adequado.

NoSQL poderia ser útil em partes complementares, especialmente onde os dados são menos estruturados ou crescem em grande volume, como logs, avaliações, preferências e cache de relatórios.

### 14. REFERÊNCIAS

- [1] CODD, Edgar F. A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, v. 13, n. 6, p. 377-387, 1970.
- [2] DATE, Christopher J. Introdução a Sistemas de Bancos de Dados. 8. ed. Rio de Janeiro: Elsevier, 2004.
- [3] HAERDER, Theo; REUTER, Andreas. Principles of Transaction-Oriented Database Recovery. ACM Computing Surveys, v. 15, n. 4, p. 287-317, 1983.
- [4] BREWER, Eric. A Certain Freedom: Thoughts on the CAP Theorem. Proceedings of the 29th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, 2010.
- [5] ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de Banco de Dados. 6. ed. São Paulo: Pearson, 2011.
- [6] MYSQL. MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: 21 mai. 2026.